Task1_ML_Classification.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

[4] ✓ 0s

```
# Check for missing values
print(df.isnull().sum())
```

```
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
target               0
dtype: int64
```

[5] ✓ 0s

```
# Dataset information
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
 4   target             150 non-null    int64
dtypes: float64(4), int64(1)
memory usage: 6.0 KB
```

Task1_ML_Classification.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

Share

[6] ✓ 0s

```
# Summary statistics
df.describe()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333	1.000000
std	0.828066	0.435866	1.765298	0.762238	0.819232
min	4.300000	2.000000	1.000000	0.100000	0.000000
25%	5.100000	2.800000	1.600000	0.300000	0.000000
50%	5.800000	3.000000	4.350000	1.300000	1.000000
75%	6.400000	3.300000	5.100000	1.800000	2.000000
max	7.900000	4.400000	6.900000	2.500000	2.000000

Task1_ML_Classification.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

[11] 0s

```
from sklearn.model_selection import train_test_split

# Features (X) and Target (y)
X = df.drop('target', axis=1)
y = df['target']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

print("Training data shape:", X_train.shape)
print("Testing data shape:", X_test.shape)
```

```
Training data shape: (120, 4)
Testing data shape: (30, 4)
```

[12] 1s

```
from sklearn.linear_model import LogisticRegression

# Create model
lr_model = LogisticRegression(max_iter=200)

# Train model
lr_model.fit(X_train, y_train)

# Predictions
y_pred_lr = lr_model.predict(X_test)

print("Model trained successfully!")
```

```
Model trained successfully!
```

Task1_ML_Classification.ipynb

File Edit View Insert Runtime Tools Help

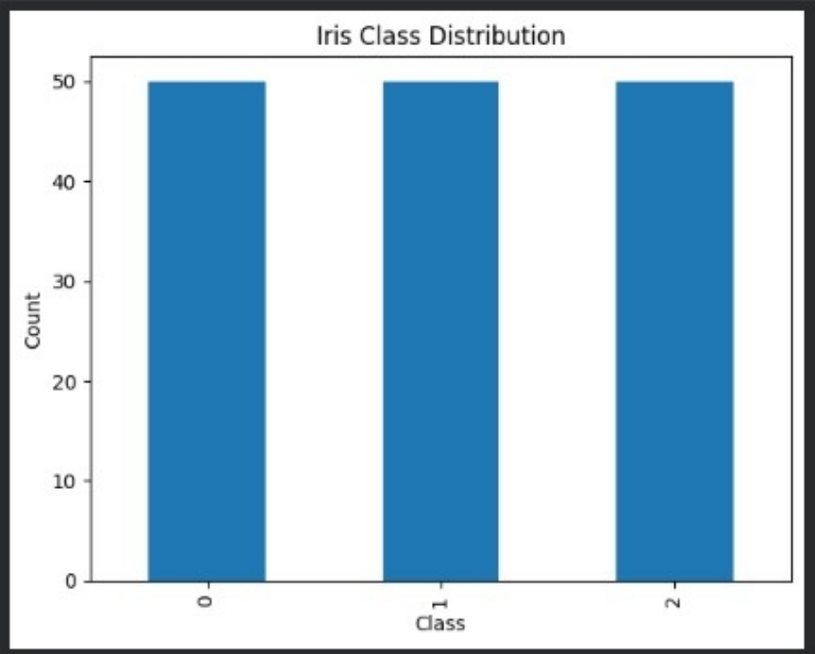
Commands + Code + Text Run all

RAM Disk

Share

[10] ✓ 0s

```
df['target'].value_counts().plot(kind='bar')
plt.xlabel('Class')
plt.ylabel('Count')
plt.title('Iris Class Distribution')
plt.show()
```



Task1_ML_Classification.ipynb

File Edit View Insert Runtime Tools Help

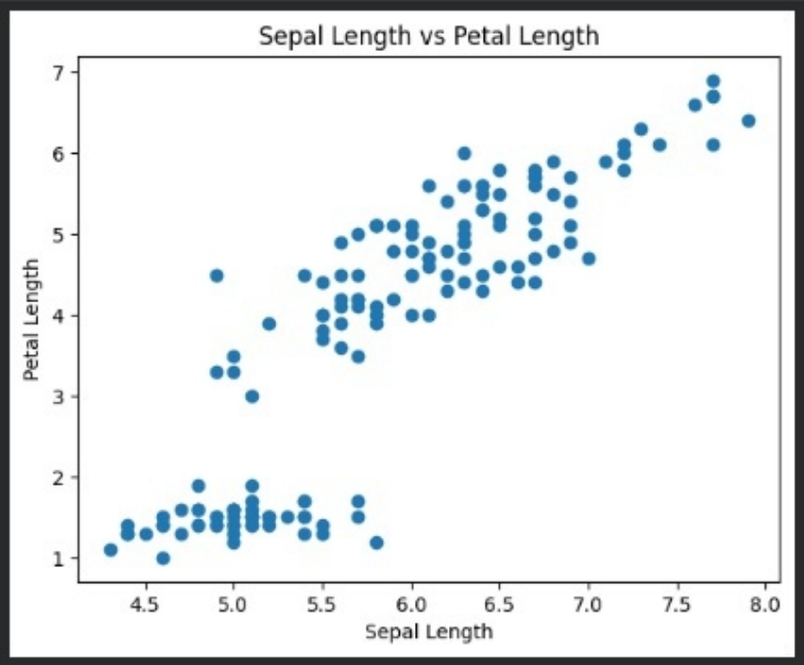
Commands + Code + Text Run all

RAM Disk

Share

[9] ✓ 0s

```
plt.scatter(df['sepal length (cm)'], df['petal length (cm)'])
plt.xlabel('Sepal Length')
plt.ylabel('Petal Length')
plt.title('Sepal Length vs Petal Length')
plt.show()
```



Task1_ML_Classification.ipynb

File Edit View Insert Runtime Tools Help

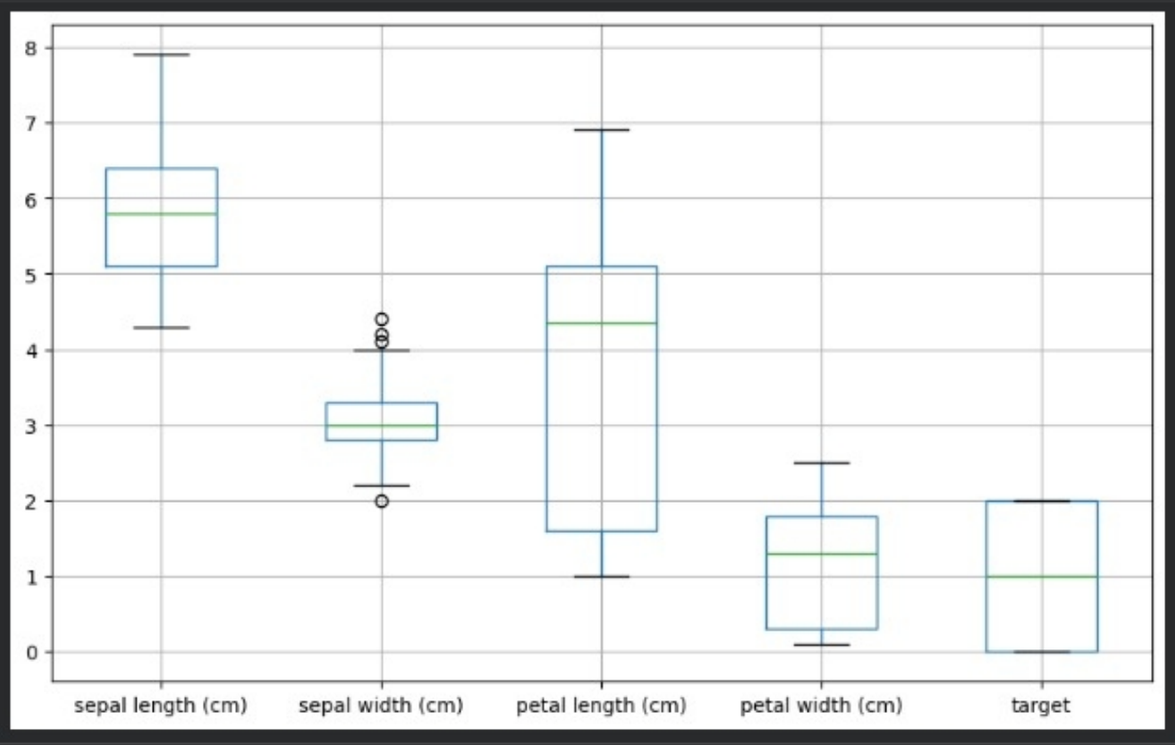
Share

Commands + Code + Text Run all

RAM Disk

[8] 0s

```
df.boxplot(figsize=(10,6))
plt.show()
```



Task1_ML_Classification.ipynb

File Edit View Insert Runtime Tools Help

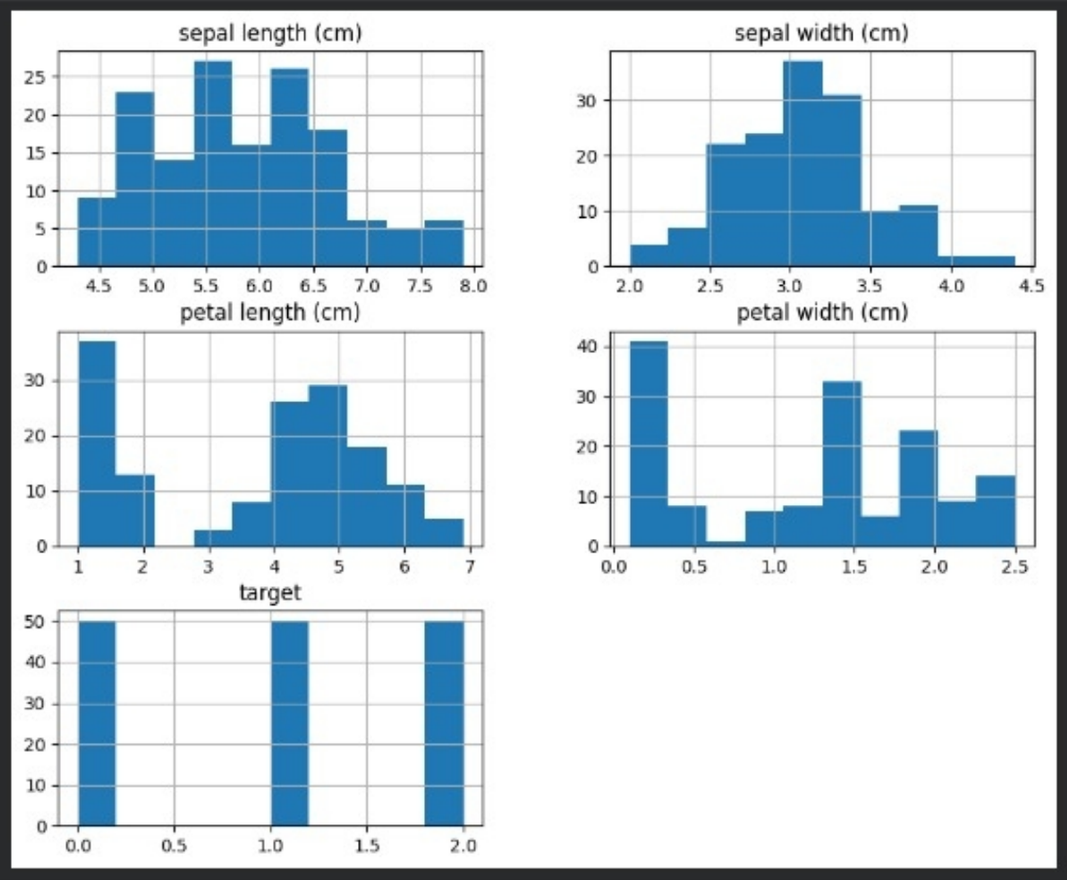
Commands + Code + Text Run all

RAM Disk

Share

[7] 0s

```
df.hist(figsize=(10,8))
plt.show()
```



Task1_ML_Classification.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

Share

[13] ✓ 0s

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

accuracy = accuracy_score(y_test, y_pred_lr)
precision = precision_score(y_test, y_pred_lr, average='weighted')
recall = recall_score(y_test, y_pred_lr, average='weighted')
f1 = f1_score(y_test, y_pred_lr, average='weighted')

print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)
```

```
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1 Score: 1.0
```

Task1_ML_Classification.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

Share

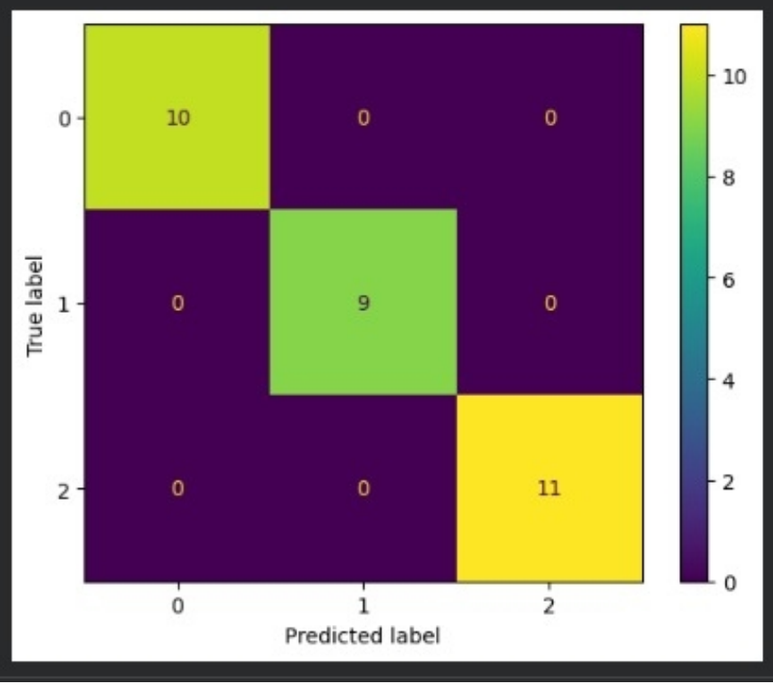
[14] ✓ 0s

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

cm = confusion_matrix(y_test, y_pred_lr)

disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()

plt.show()
```



Task1_ML_Classification.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

[20] 0s

```
comparison = pd.DataFrame({
    'Model': ['Logistic Regression', 'Random Forest'],
    'Accuracy': [accuracy, accuracy_rf],
    'Precision': [precision, precision_rf],
    'Recall': [recall, recall_rf],
    'F1 Score': [f1, f1_rf]
})

comparison
```

	Model	Accuracy	Precision	Recall	F1 Score
0	Logistic Regression	1.0	1.0	1.0	1.0
1	Random Forest	1.0	1.0	1.0	1.0

Next steps: Generate code with comparison | New interactive sheet

Task1_ML_Classification.ipynb

File Edit View Insert Runtime Tools Help

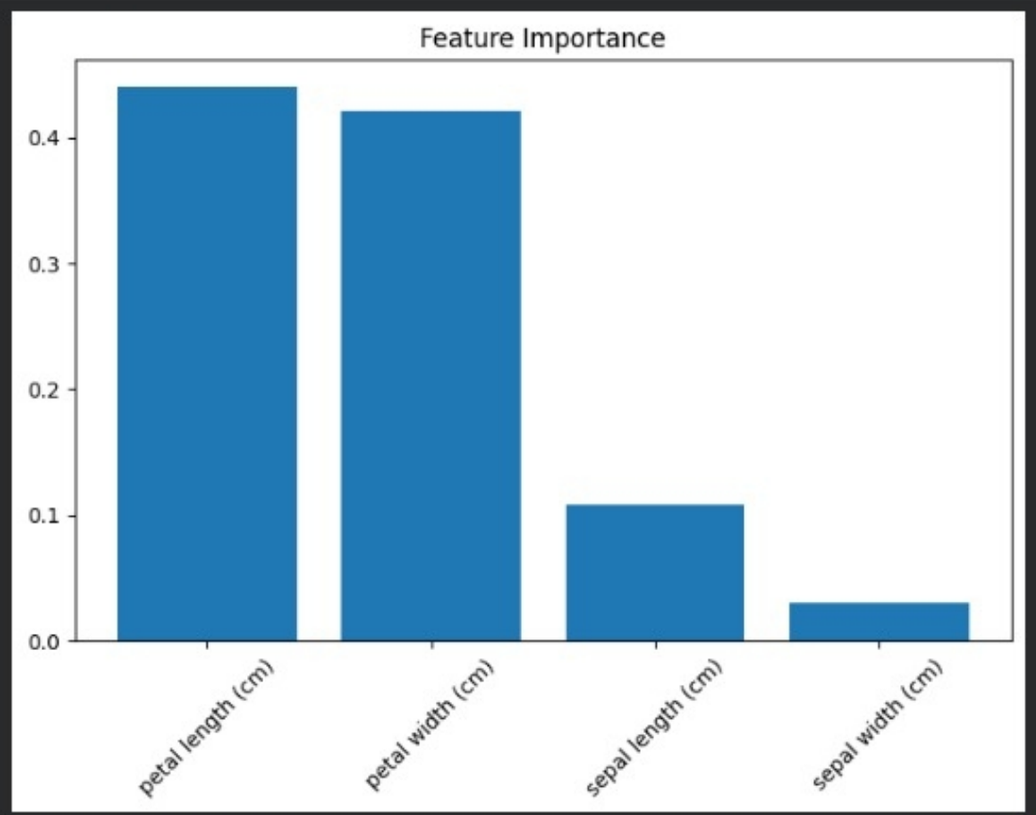
Commands + Code + Text Run all

RAM Disk

Share

[19] ✓ 0s

```
        feature_importance['Importance'])
plt.xticks(rotation=45)
plt.title('Feature Importance')
plt.show()
```



Task1_ML_Classification.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

Share

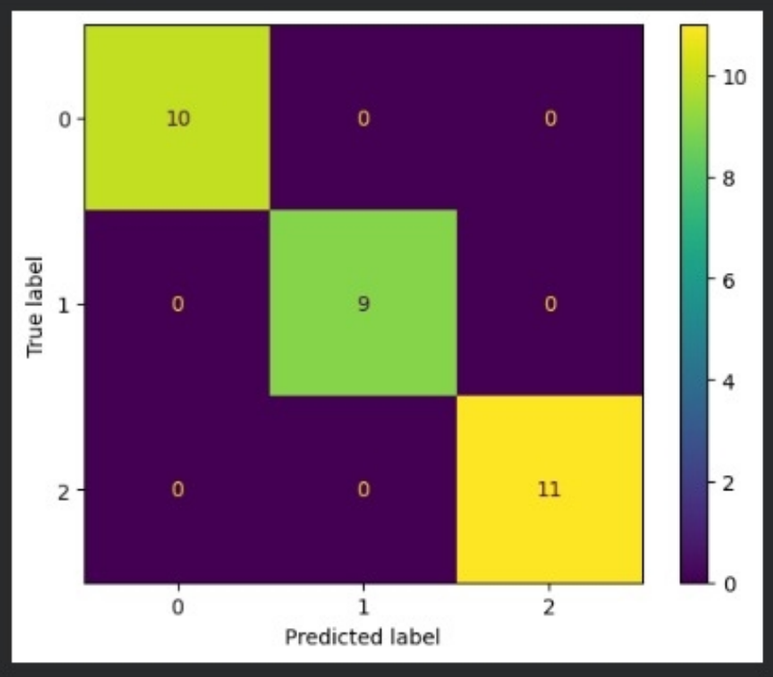
[18] ✓ 0s

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

cm_rf = confusion_matrix(y_test, y_pred_rf)

disp = ConfusionMatrixDisplay(confusion_matrix=cm_rf)
disp.plot()

plt.show()
```



Task1_ML_Classification.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

Share

[17] ✓ 0s

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

accuracy_rf = accuracy_score(y_test, y_pred_rf)
precision_rf = precision_score(y_test, y_pred_rf, average='weighted')
recall_rf = recall_score(y_test, y_pred_rf, average='weighted')
f1_rf = f1_score(y_test, y_pred_rf, average='weighted')

print("Accuracy:", accuracy_rf)
print("Precision:", precision_rf)
print("Recall:", recall_rf)
print("F1 Score:", f1_rf)
```

```
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1 Score: 1.0
```

Task1_ML_Classification.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

Share

[15] ✓ 0s

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred_lr))
```

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      1.00      1.00         9
           2       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

[16] ✓ 1s

```
from sklearn.ensemble import RandomForestClassifier

# Create model
rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

# Train model
rf_model.fit(X_train, y_train)

# Predictions
y_pred_rf = rf_model.predict(X_test)

print("Random Forest trained successfully!")
```

```
Random Forest trained successfully!
```

Task1_ML_Classification.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

Share

[21] ✓ 0s

```
plt.plot([0,1], [0,1], linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend()
plt.show()
```